Universidad Sergio Arboleda

Parcial 2

Fundamentos de Inteligencia Artificial

Algoritmos genéticos, ascenso de colinas, temple simulado, ACO y PSO

Estudiante: Lina María Castañeda Hernández

Fecha: 14 de octubre de 2026

Soluciones matemáticas, análisis conceptual, implementaciones en Python y resultados reproducibles

Informe elaborado conforme a los cinco puntos del documento *Parcial 2*. Los scripts originales se incluyen en la carpeta `codigo/`; sus salidas, en `salidas/`.

Índice

Introducción y metodología **2**

1. Punto 1. Algoritmos evolutivos y algoritmos genéticos **2**
1.1. A. Componente matemático y conceptual . . . 2
1.1.1. A.1. Decodificación y aptitud . . . 2
1.1.2. A.2. Selección por ruleta . . . 2
1.1.3. A.3. Cruce de un punto . . . 2
1.1.4. A.4. Mutación del primer descendiente . . . 2
1.1.5. A.5. Interpretación de los operadores . . . 3
1.2. B. Implementación en Python . . . 3

2. Punto 2. Ascenso de colinas y óptimos locales **5**
2.1. A. Componente matemático y conceptual . . . 5
2.1.1. A.1. Ejecución desde $x_0 = 0$ . . . 5
2.1.2. A.2. Ejecución desde $x_0 = 4$ . . . 5
2.1.3. A.3. Máximos locales y global . . . 5
2.1.4. A.4. Influencia del inicio . . . 5
2.1.5. A.5. Propuesta: reinicios aleatorios guiados por memoria . . . 6
2.2. B. Implementación en Python . . . 6

3. Punto 3. Temple simulado y aceptación probabilística **7**
3.1. A. Componente matemático y conceptual . . . 7
3.1.1. A.1. Cálculos de las tres transiciones . . . 7
3.1.2. A.2. Efecto de la temperatura . . . 7
3.1.3. A.3. Enfriamiento demasiado rápido . . . 8
3.1.4. A.4. Comparación con ascenso de colinas . . . 8
3.2. B. Implementación en Python . . . 8

4. Punto 4. ACO: rutas y feromonas **10**
4.1. A. Componente matemático y conceptual . . . 10
4.1.1. A.1. Pesos y probabilidades . . . 10
4.1.2. A.2. Alternativa más probable . . . 10
4.1.3. A.3. Evaporación y depósito . . . 10
4.1.4. A.4. Función de la evaporación y riesgo de dominio de feromonas . . . 10
4.2. B. Implementación en Python . . . 11

5. Punto 5. PSO: movimiento y memoria **13**
5.1. A. Componente matemático y conceptual . . . 13
5.1.1. A.1. Inercia, componente cognitivo y componente social . . . 13
5.1.2. A.2. Nueva velocidad, posición y aptitud . . . 13
5.1.3. A.3. Actualización de *pbest* y *gbest* . . . 13
5.1.4. A.4. Mejor personal frente a mejor global . . . 14
5.1.5. A.5. Intercambio de información: PSO frente a ACO . . . 14
5.2. B. Implementación de PSO en dos dimensiones . . . 14

Conclusiones generales **16**

Archivos y reproducibilidad **16**

Introducción y metodología

Este informe resuelve los cinco problemas del parcial, respetando sus componentes A (matemático y conceptual) y B (implementación). Se utilizaron los archivos Python suministrados y se volvieron a ejecutar con las semillas establecidas para contrastar los resultados con las salidas entregadas. Las figuras se construyeron a partir de las trazas de ejecución. La propuesta del punto 2.A.5 es *conceptual*; no altera el código obligatorio del apartado B. Los resultados de las ejecuciones con semillas fijas ilustran el comportamiento observado, sin constituir garantías generales de convergencia.

1. Punto 1. Algoritmos evolutivos y algoritmos genéticos

Se maximiza $f(x) = x^2$, con $x \in \{0, 1, \ldots, 31\}$ y cromosomas binarios de cinco bits. La población inicial es $P = \{00101, 01100, 10010, 11001\}$.

1.1. A. Componente matemático y conceptual

1.1.1. A.1. Decodificación y aptitud

Se convierte cada cadena binaria a decimal mediante las potencias de 2 correspondientes a sus cinco posiciones. Por ejemplo, $00101_2 = 4 + 1 = 5$, y su aptitud es $5^2 = 25$.

Tabla 1: Población inicial y aptitud

Cromosoma	Valor decimal x	$f(x) = x^2$
00101	5	25
01100	12	144
10010	18	324
11001	25	625

1.1.2. A.2. Selección por ruleta

La aptitud total es $\sum_i f(x_i) = 25 + 144 + 324 + 625 = 1118$. La probabilidad de selección del cromosoma i es $p_i = f(x_i)/1118$.

$$p_{00101} = 25/1118 \approx 0{,}02236 = 2{,}24\,\%, \qquad p_{01100} = 144/1118 \approx 0{,}12880 = 12{,}88\,\%,$$
$$p_{10010} = 324/1118 \approx 0{,}28980 = 28{,}98\,\%, \qquad p_{11001} = 625/1118 \approx 0{,}55903 = 55{,}90\,\%.$$

La ruleta favorece individuos más aptos, pero no garantiza que el cromosoma de mayor aptitud sea seleccionado en cada sorteo.

1.1.3. A.3. Cruce de un punto

Se seleccionan 01100 y 11001, cortando después del segundo bit:

$$01 \mid 100, \qquad 11 \mid 001.$$

Intercambiando los segmentos derechos:

$$\boxed{01 \mid 001 = 01001_2 = 9, \quad f(9) = 81}, \qquad \boxed{11 \mid 100 = 11100_2 = 28, \quad f(28) = 784}.$$

1.1.4. A.4. Mutación del primer descendiente

El primer descendiente es 01001. Al invertir su último bit se obtiene $01000_2 = 8$, con $f(8) = 64$. Antes de mutar, su aptitud era $f(9) = 81$; el cambio inmediato es $64 - 81 = -17$.

1.1.5. A.5. Interpretación de los operadores

Un algoritmo genético es un tipo de algoritmo evolutivo: ambos utilizan una población de soluciones que evoluciona mediante selección y variación, mientras que el algoritmo genético emplea habitualmente representaciones de cromosomas y operadores de cruce y mutación. La **selección** incrementa la participación reproductiva de soluciones aptas; el **cruce** recombina material de dos padres; y la **mutación** introduce variabilidad al modificar algunos bits. Una mutación puede empeorar la aptitud inmediata y, aun así, resultar útil porque permite explorar configuraciones que no estaban presentes y ayuda a evitar la pérdida prematura de diversidad. No garantiza por sí sola mejoras posteriores.

1.2. B. Implementación en Python

El programa utiliza selección por ruleta, cruce de un punto con probabilidad 0,8, mutación independiente por bit con probabilidad 0,05, elitismo de un individuo, población de tamaño cuatro, veinte generaciones y semilla 42. La función `decodificar` convierte cromosomas a enteros; `aptitud` calcula x^2; `seleccion_ruleta` hace selección proporcional a aptitud; `cruce_un_punto` recombina las cadenas; `mutar` considera cada bit independientemente; y `algoritmo_genetico` coordina las generaciones. El mejor cromosoma se copia sin cambios a la nueva población antes de crear los demás descendientes.

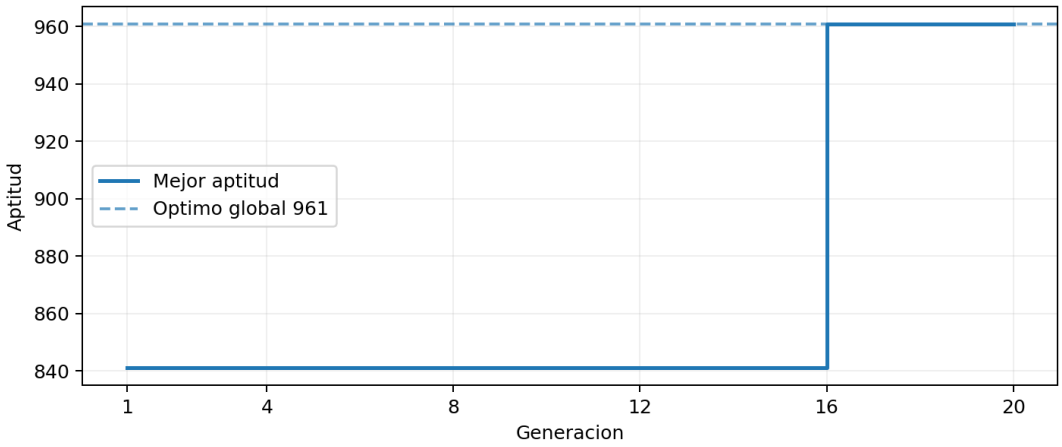


Figura 1: Mejor aptitud por generación del algoritmo genético; óptimo teórico de 961.

Tabla 2: Síntesis de la ejecución del algoritmo genético

Indicador	Resultado
Mejor cromosoma final	`11111`
Valor decimal	31
Mejor aptitud	961
Generaciones 1–15	mejor aptitud 841
Generaciones 16–20	mejor aptitud 961
Óptimo matemático	$f(31) = 961$

El historial completo fue

$$[\underbrace{841, \ldots, 841}_{\text{15 generaciones}}, \underbrace{961, \ldots, 961}_{\text{5 generaciones}}].$$

Interpretación. Sí se encontró el óptimo global: la función x^2 es creciente en el dominio no negativo $0 \leq x \leq 31$ y alcanza su máximo 961 en $x = 31$, representado por `11111`. Sin embargo, veinte generaciones no garantizan alcanzar ese cromosoma en todas las ejecuciones: la selección, el cruce y la mutación son estocásticos y pueden no producir la combinación óptima dentro del presupuesto de iteraciones. El elitismo conserva lo mejor ya descubierto, pero no garantiza descubrir el máximo.

Código completo del punto 1

```python
"""Punto 1 - Algoritmo genetico para maximizar f(x) = x^2, x en {0,...,31}."""
import random

N_BITS = 5
P_CRUCE = 0.8
P_MUT = 0.05
GENERACIONES = 20
SEMILLA = 42


def decodificar(crom):
    return int(crom, 2)


def aptitud(crom):
    return decodificar(crom) ** 2


def seleccion_ruleta(poblacion):
    aptitudes = [aptitud(c) for c in poblacion]
    total = sum(aptitudes)
    if total == 0:
        return random.choice(poblacion)
    r = random.uniform(0, total)
    acumulado = 0
    for crom, f in zip(poblacion, aptitudes):
        acumulado += f
        if r <= acumulado:
            return crom
    return poblacion[-1]


def cruce_un_punto(p1, p2):
    if random.random() < P_CRUCE:
        punto = random.randint(1, N_BITS - 1)
        return p1[:punto] + p2[punto:], p2[:punto] + p1[punto:]
    return p1, p2


def mutar(crom):
    bits = list(crom)
    for i in range(len(bits)):
        if random.random() < P_MUT:
            bits[i] = '1' if bits[i] == '0' else '0'
    return ''.join(bits)


def algoritmo_genetico(poblacion):
    random.seed(SEMILLA)
    historial = []
    for gen in range(1, GENERACIONES + 1):
        elite = max(poblacion, key=aptitud)          # elitismo
        nueva = [elite]
        while len(nueva) < len(poblacion):
            p1 = seleccion_ruleta(poblacion)
            p2 = seleccion_ruleta(poblacion)
            h1, h2 = cruce_un_punto(p1, p2)
            nueva.append(mutar(h1))
            if len(nueva) < len(poblacion):
                nueva.append(mutar(h2))
        poblacion = nueva
        mejor = max(poblacion, key=aptitud)
```

```
        historial.append(aptitud(mejor))
        print(f"Gen {gen:2d}: poblacion={poblacion} mejor={mejor} "
              f"x={decodificar(mejor)} f={aptitud(mejor)}")
    mejor = max(poblacion, key=aptitud)
    return mejor, historial


if __name__ == "__main__":
    inicial = ["00101", "01100", "10010", "11001"]
    print("Poblacion inicial:")
    total = sum(aptitud(c) for c in inicial)
    for c in inicial:
        print(f"  {c} -> x={decodificar(c)}, f={aptitud(c)}, p={aptitud(c)/total:.4f}")
    mejor, hist = algoritmo_genetico(inicial)
    print("\nMejor cromosoma final:", mejor)
    print("x =", decodificar(mejor), " aptitud =", aptitud(mejor))
    print("Historial mejor aptitud:", hist)
```

Listing 1: Implementación original del algoritmo genético

2. Punto 2. Ascenso de colinas y óptimos locales

Se maximiza la función discreta siguiente y se consideran solamente los vecinos $x-1$ y $x+1$ que estén dentro del dominio:

x	0	1	2	3	4	5	6	7	8
$f(x)$	1	4	7	5	3	6	9	8	5

2.1. A. Componente matemático y conceptual

2.1.1. A.1. Ejecución desde $x_0 = 0$

Desde $x = 0$, el único vecino es 1 y $f(1) = 4 > f(0) = 1$; se avanza a 1. Desde 1, se selecciona 2, pues $f(2) = 7 > f(0) = 1$, y mejora respecto a $f(1) = 4$. En 2, sus vecinos 1 y 3 tienen aptitudes 4 y 5, inferiores a 7. Se detiene:

$$\boxed{0 \to 1 \to 2, \quad x_{\text{final}} = 2, \quad f(2) = 7}.$$

2.1.2. A.2. Ejecución desde $x_0 = 4$

En 4, los vecinos 3 y 5 presentan aptitudes 5 y 6: se elige 5. Desde 5, el vecino 6 tiene aptitud 9, mayor que $f(5) = 6$. En 6, los vecinos tienen aptitudes 6 y 8, ambas menores que 9. Así:

$$\boxed{4 \to 5 \to 6, \quad x_{\text{final}} = 6, \quad f(6) = 9}.$$

2.1.3. A.3. Máximos locales y global

El estado $x = 2$ es un máximo local, ya que $f(2) = 7 > f(1) = 4$ y $f(2) = 7 > f(3) = 5$. El estado $x = 6$ también es máximo local y es el **máximo global**, pues $f(6) = 9$ es el mayor valor de toda la tabla.

2.1.4. A.4. Influencia del inicio

El método es codicioso y solo admite mejoras estrictas. Desde $0, 1, 2, 3$ termina en $x = 2$; desde $4, 5, 6, 7, 8$ termina en $x = 6$. El inicio determina la región del espacio de búsqueda que se explora y puede conducir a máximos diferentes, porque el algoritmo no atraviesa descensos temporales para salir de un máximo local.

2.1.5. A.5. Propuesta: reinicios aleatorios guiados por memoria

Se propone ejecutar varias búsquedas desde puntos de partida seleccionados aleatoriamente *sin repetir los inicios ya utilizados*. La memoria almacena esos estados iniciales y la mejor solución hallada. Cada búsqueda individual mantiene la regla original de ascenso de colinas: seleccionar el mejor vecino y moverse solo ante una mejora estricta. Los reinicios favorecen la exploración de regiones distintas, y la memoria evita repetir exactamente una búsqueda determinista desde el mismo inicio. Esto no impide que diferentes inicios desemboquen en un mismo máximo local ni garantiza el óptimo con pocos intentos. Si hay n inicios posibles, el almacenamiento de los ya utilizados requiere hasta $O(n)$ espacio. Para k ejecuciones de costo individual T, el trabajo de búsqueda cuesta aproximadamente $O(kT)$, además de seleccionar y administrar los inicios no utilizados. **Esta propuesta es exclusivamente conceptual y no modifica la implementación obligatoria del apartado B.**

2.2. B. Implementación en Python

La función `ascenso_colinas(valores, inicio)` registra el inicio, obtiene vecinos válidos, escoge el de mayor aptitud con `max(..., key=...)` y avanza solo si la mejora es estricta. En caso contrario, devuelve el estado final y la trayectoria.

Tabla 3: Ejecuciones desde los nueve estados iniciales

Inicio	Trayectoria	Final	f(final)
0	$0 \to 1 \to 2$	2	7
1	$1 \to 2$	2	7
2	2	2	7
3	$3 \to 2$	2	7
4	$4 \to 5 \to 6$	6	9
5	$5 \to 6$	6	9
6	6	6	9
7	$7 \to 6$	6	9
8	$8 \to 7 \to 6$	6	9

Cinco de los nueve inicios llegan al máximo global, por lo que la proporción es

$$\boxed{5/9 \approx 0{,}5556 = 55{,}56\,\%}.$$

Relación con reinicios aleatorios. Si cada inicio se elige de forma independiente y uniforme entre los nueve estados, cada ejecución tiene probabilidad 5/9 de encontrar el máximo global. Con k reinicios independientes *con reemplazo*, la probabilidad de encontrarlo al menos una vez es $1 - (4/9)^k$. En la propuesta con memoria y *sin reemplazo*, esa fórmula de independencia no aplica directamente; su ventaja es evitar repetir inicios, y al agotar los nueve inicios necesariamente se habrá evaluado el estado $x = 6$ en este problema concreto.

Código completo del punto 2

```python
"""Punto 2 - Ascenso de colinas en un espacio discreto."""

VALORES = [1, 4, 7, 5, 3, 6, 9, 8, 5]   # f(x) para x = 0..8


def ascenso_colinas(valores, inicio):
    """Devuelve el estado final y la trayectoria."""
    actual = inicio
    trayectoria = [actual]
```

```
    while True:
        vecinos = [v for v in (actual - 1, actual + 1) if 0 <= v < len(valores)]
        mejor_vecino = max(vecinos, key=lambda v: valores[v])
        if valores[mejor_vecino] > valores[actual]:   # solo mejora estricta
            actual = mejor_vecino
            trayectoria.append(actual)
        else:
            return actual, trayectoria


if __name__ == "__main__":
    maximo_global = max(VALORES)
    exitos = 0
    print(f"{'Inicio':>6} | {'Trayectoria':<18} | {'Final':>5} | {'f(final)':>8}")
    print("-" * 48)
    for inicio in range(len(VALORES)):
        final, tray = ascenso_colinas(VALORES, inicio)
        tray_txt = " -> ".join(map(str, tray))
        print(f"{inicio:>6} | {tray_txt:<18} | {final:>5} | {VALORES[final]:>8}")
        if VALORES[final] == maximo_global:
            exitos += 1
    print(f"\nInicios que llegan al maximo global (f={maximo_global}): "
          f"{exitos}/{len(VALORES)} = {exitos/len(VALORES):.2%}")
```

Listing 2: Implementación original de ascenso de colinas

3. Punto 3. Temple simulado y aceptación probabilística

Se minimiza $E(x) = x^2$, $x \in \{-5, \ldots, 5\}$. La regla de aceptación, con $\Delta E = E(x_{\text{nuevo}}) - E(x_{\text{actual}})$ y $T > 0$, es

$$P(\text{aceptar}) = \begin{cases} 1, & \Delta E \leq 0, \\ e^{-\Delta E/T}, & \Delta E > 0. \end{cases}$$

Una solución peor se acepta si el valor uniforme proporcionado cumple $u < P$.

3.1. A. Componente matemático y conceptual

3.1.1. A.1. Cálculos de las tres transiciones

Caso A. Actual $x = 3$, candidato $x = 2$, $T = 5$, $u = 0{,}70$: $E(3) = 9$, $E(2) = 4$, $\Delta E = 4 - 9 = -5$. Como mejora la energía, $P = 1$ y se **acepta**.

Caso B. Actual $x = 2$, candidato $x = 3$, $T = 5$, $u = 0{,}30$: $\Delta E = 9 - 4 = 5 > 0$; $P = e^{-5/5} = e^{-1} \approx 0{,}367879$. Como $0{,}30 < 0{,}367879$, se **acepta** el empeoramiento.

Caso C. Igual transición, pero $T = 1$, $u = 0{,}30$: $\Delta E = 5$ y $P = e^{-5} \approx 0{,}006738$. Como $0{,}30 > 0{,}006738$, se **rechaza**; permanece en $x = 2$.

Tabla 4: Comparación de las tres decisiones

Caso	Actual	Candidato	ΔE	P	Decisión
A	3	2	-5	1	Aceptar
B	2	3	5	0,367879	Aceptar
C	2	3	5	0,006738	Rechazar

3.1.2. A.2. Efecto de la temperatura

Para un aumento fijo $\Delta E > 0$, $e^{-\Delta E/T}$ crece con T. Una temperatura alta permite aceptar empeoramientos más frecuentemente y favorece la exploración; al disminuir T, la aceptación de

soluciones peores se reduce y la búsqueda se concentra en las soluciones prometedoras. Para $\Delta E = 5$, la probabilidad es 36,79 % con $T = 5$ y apenas 0,674 % con $T = 1$.

3.1.3. A.3. Enfriamiento demasiado rápido

Si la temperatura decrece precipitadamente, la aceptación de aumentos de energía puede volverse prácticamente nula antes de explorar suficientemente el dominio. Esto favorece la convergencia prematura hacia mínimos locales. Un enfriamiento gradual prolonga la exploración, pero no garantiza el óptimo global. En el programa se utiliza $T_{k+1} = 0{,}90T_k$.

3.1.4. A.4. Comparación con ascenso de colinas

El ascenso de colinas del punto 2 selecciona el mejor vecino y solo avanza si mejora estrictamente la aptitud; para comparar bajo minimización equivaldría a un descenso de colinas que rechaza empeoramientos. El temple simulado escoge un vecino aleatorio, acepta mejoras (o igualdades) siempre y acepta empeoramientos con probabilidad $e^{-\Delta E/T}$. Esta flexibilidad puede ayudarlo a escapar de óptimos locales. La temperatura regula el balance exploración–explotación.

3.2. B. Implementación en Python

Se parte de $x_0 = 5$, $T_0 = 10$, cien iteraciones, vecinos aleatorios válidos $x \pm 1$, enfriamiento geométrico del 10 % y semilla 42. Se guardan el estado actual, la mejor solución y las energías de ambos en cada iteración.

Tabla 5: Evolución inicial y resultado final del temple simulado

Iteración	Temperatura	Actual	E. actual	Mejor	E. mejor
1	10.00000	4	16	4	16
2	9.00000	3	9	3	9
3	8.10000	4	16	3	9
5	6.56100	2	4	2	4
6	5.90490	1	1	1	1
9	4.30467	0	0	0	0
10	3.87420	−1	1	0	0
100	0.00030	0	0	0	0

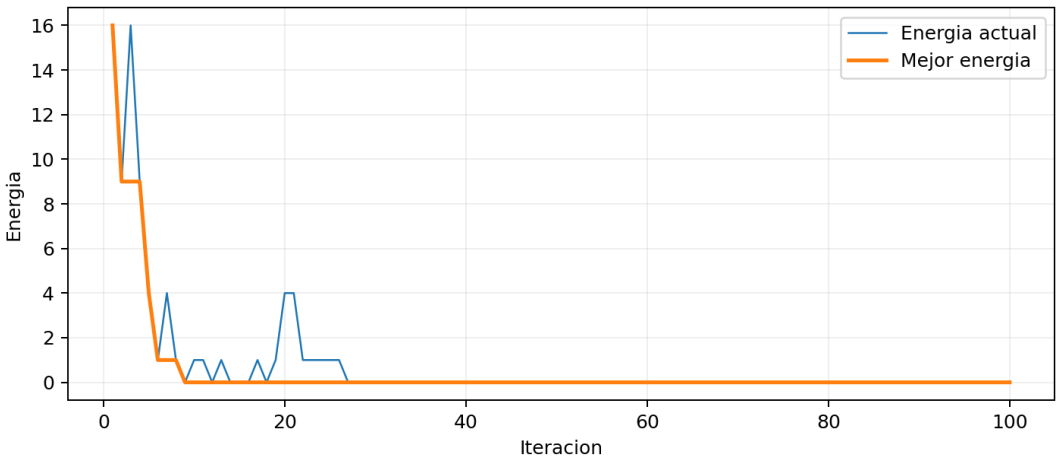


Figura 2: Energía actual y mejor energía acumulada por iteración.

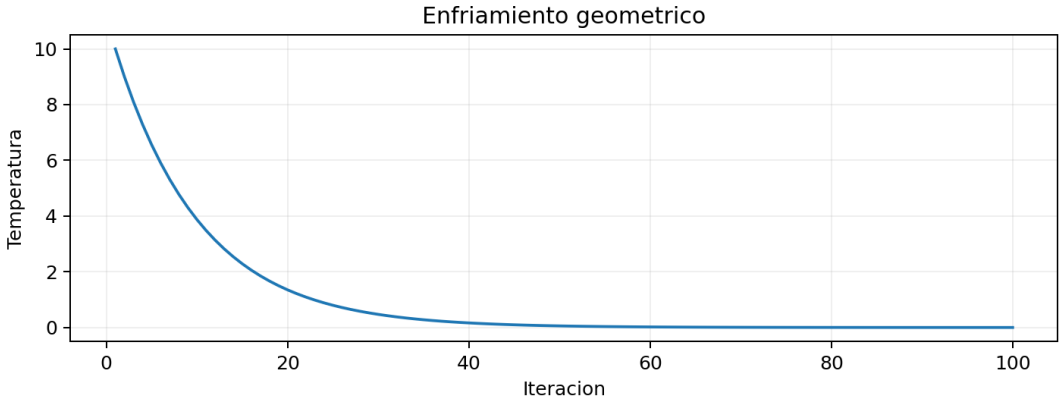


Figura 3: Descenso geométrico de la temperatura durante las cien iteraciones.

La mejor solución hallada es $x = 0$, con energía mínima $E(0) = 0$; aparece por primera vez en la iteración 9. El historial evidencia que, tras encontrar el óptimo, en la iteración 10 el estado actual se desplazó a $x = -1$, cuya energía es 1. Por ello es indispensable conservar por separado la mejor solución: el *estado actual* puede empeorar al aceptar movimientos exploratorios, mientras que el registro *mejor* evita perder la mejor energía alcanzada.

Código completo del punto 3

```python
"""Punto 3 - Temple simulado para minimizar E(x) = x^2, x en {-5,...,5}."""
import math
import random

X_MIN, X_MAX = -5, 5
X0 = 5
T0 = 10.0
ALFA = 0.90
ITERACIONES = 100
SEMILLA = 42


def energia(x):
    return x ** 2


def vecino_aleatorio(x):
    vecinos = [v for v in (x - 1, x + 1) if X_MIN <= v <= X_MAX]
    return random.choice(vecinos)


def temple_simulado():
    random.seed(SEMILLA)
    actual, T = X0, T0
    mejor = actual
    historial = []   # (iteracion, T, x_actual, E_actual, mejor, E_mejor)
    for k in range(1, ITERACIONES + 1):
        candidato = vecino_aleatorio(actual)
        dE = energia(candidato) - energia(actual)
        if dE <= 0:
            actual = candidato
        else:
            u = random.random()
            if u < math.exp(-dE / T):
                actual = candidato
        if energia(actual) < energia(mejor):
            mejor = actual
        historial.append((k, T, actual, energia(actual), mejor, energia(mejor)))
        T *= ALFA
    return mejor, historial


```

```python
if __name__ == "__main__":
    # Parte A: los tres casos
    for nombre, xa, xc, T, u in [("A", 3, 2, 5, 0.70), ("B", 2, 3, 5, 0.30), ("C", 2, 3,
        1, 0.30)]:
        dE = energia(xc) - energia(xa)
        p = 1.0 if dE <= 0 else math.exp(-dE / T)
        decision = "acepta" if (dE <= 0 or u < p) else "rechaza"
        print(f"Caso {nombre}: dE={dE}, P={p:.4f}, u={u} -> {decision}")

    mejor, hist = temple_simulado()
    print(f"\n{'k':>3} {'T':>9} {'x':>3} {'E':>3} {'mejor':>5} {'E_mejor':>7}")
    for k, T, x, E, m, Em in hist:
        print(f"{k:>3} {T:>9.5f} {x:>3} {E:>3} {m:>5} {Em:>7}")
    print(f"\nMejor solucion: x = {mejor}, energia minima = {energia(mejor)}")
```

Listing 3: Implementación original del temple simulado

4. Punto 4. ACO: rutas y feromonas

4.1. A. Componente matemático y conceptual

Una hormiga se encuentra en A y puede viajar a B, C o D. Se dispone de $(d_{AB}, \tau_{AB}) = (2, 1)$, $(d_{AC}, \tau_{AC}) = (4, 2)$, $(d_{AD}, \tau_{AD}) = (5, 1)$; se fijan $\alpha = 1$, $\beta = 2$ y $\eta_{Aj} = 1/d_{Aj}$.

4.1.1. A.1. Pesos y probabilidades

Los pesos son $w_{Aj} = \tau_{Aj}^{\alpha} \eta_{Aj}^{\beta}$:

$$w_{AB} = 1(1/2)^2 = 0{,}25, \qquad w_{AC} = 2(1/4)^2 = 0{,}125, \qquad w_{AD} = 1(1/5)^2 = 0{,}04.$$

Su suma es $W = 0{,}25 + 0{,}125 + 0{,}04 = 0{,}415$. Por normalización:

$$\begin{aligned} p_{AB} &= 0{,}25/0{,}415 \approx 0{,}602410 = 60{,}24\,\%, \\ p_{AC} &= 0{,}125/0{,}415 \approx 0{,}301205 = 30{,}12\,\%, \\ p_{AD} &= 0{,}04/0{,}415 \approx 0{,}096386 = 9{,}64\,\%. \end{aligned}$$

4.1.2. A.2. Alternativa más probable

A→B es la alternativa con mayor probabilidad (60,24 %), pero no está garantizada: A→C y A→D tienen probabilidades positivas. La probabilidad de elegir alguna de esas alternativas es $1 - 0{,}602410 \approx 39{,}76\,\%$. La selección probabilística mantiene oportunidades de explorar rutas distintas de la más atractiva.

4.1.3. A.3. Evaporación y depósito

Con $\rho = 0{,}2$, $Q = 10$, $L = 10$ y depósito únicamente sobre A→B, se tiene $\Delta\tau_{AB} = Q/L = 1$, y los demás depósitos son cero. Aplicando $\tau' = (1 - \rho)\tau + \Delta\tau$:

$$\begin{aligned} \tau'_{AB} &= 0{,}8(1) + 1 = \boxed{1{,}8}, \\ \tau'_{AC} &= 0{,}8(2) + 0 = \boxed{1{,}6}, \\ \tau'_{AD} &= 0{,}8(1) + 0 = \boxed{0{,}8}. \end{aligned}$$

4.1.4. A.4. Función de la evaporación y riesgo de dominio de feromonas

La evaporación reduce gradualmente la influencia de depósitos antiguos y evita que las rutas no reforzadas conserven indefinidamente su feromona. Si las concentraciones de feromona dominaran excesivamente las probabilidades, las hormigas podrían seleccionar casi siempre una ruta reforzada

inicialmente, incluso siendo subóptima, y reducir la exploración de alternativas. Este realimentación puede provocar estancamiento o convergencia prematura. La evaporación contribuye al equilibrio exploración–explotación, aunque no elimina por sí misma todo riesgo de estancamiento.

4.2. B. Implementación en Python

Se trabaja con las rutas completas A-B-Z ($L = 6$), A-C-Z ($L = 8$) y A-D-Z ($L = 10$), cada una con feromona inicial 1. Durante veinte iteraciones, diez hormigas por iteración eligen con las mismas probabilidades calculadas al comienzo de la iteración. A continuación se evapora 20 % de cada feromona y cada hormiga deposita Q/L_r en la ruta elegida. Se usan $\alpha = 1$, $\beta = 2$, $Q = 10$ y semilla 42. Inicialmente, los pesos son 1/36, 1/64 y 1/100 y las probabilidades aproximadas son 52,02 %, 29,26 % y 18,73 %. Durante la primera iteración las hormigas eligieron seis veces A-B-Z, tres A-C-Z y una A-D-Z; las feromonas quedaron:

$$\tau_{ABZ}^{(1)} = 0{,}8 + 6(10/6) = 10{,}8,$$
$$\tau_{ACZ}^{(1)} = 0{,}8 + 3(10/8) = 4{,}55,$$
$$\tau_{ADZ}^{(1)} = 0{,}8 + 1(10/10) = 1{,}8.$$

Tabla 6: Resultados de ACO: veinte iteraciones y doscientas elecciones

Ruta	Frecuencia	Porcentaje	Feromona final
A-B-Z	187	93.5 %	79.2315
A-C-Z	12	6.0 %	2.3580
A-D-Z	1	0.5 %	0.0259

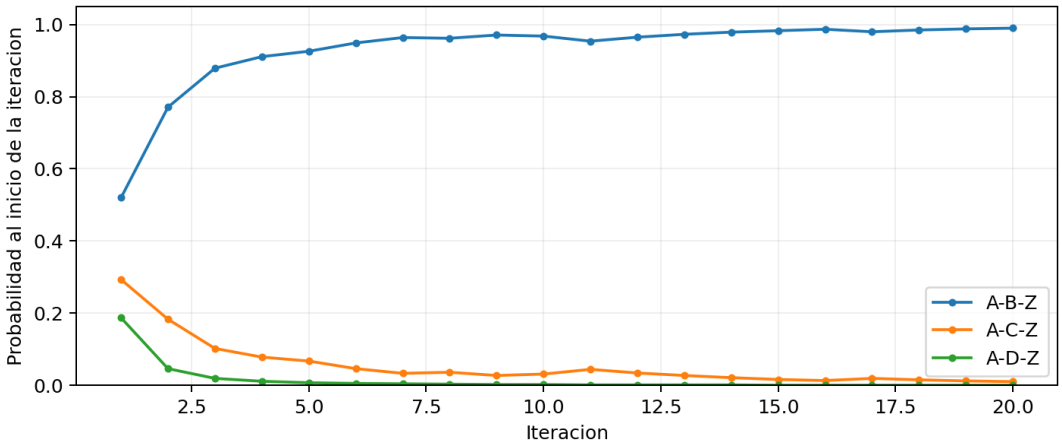


Figura 4: Evolución de probabilidades de selección al inicio de cada iteración.

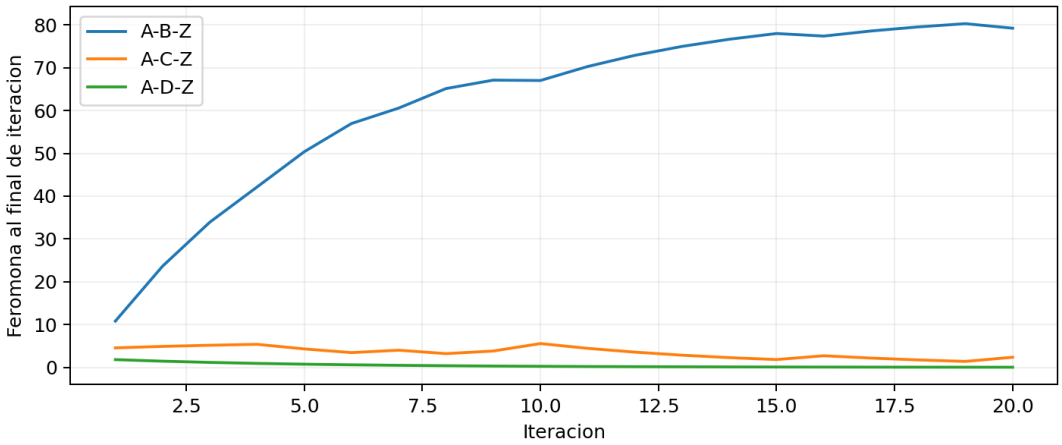


Figura 5: Evolución de las feromonas al final de cada iteración.

La mejor ruta observada fue **A-B-Z**, longitud 6. La heurística $(1/L_r)^2$ favorece las rutas más cortas, mientras que el depósito Q/L_r aporta un refuerzo mayor por hormiga sobre rutas de menor longitud. La experiencia colectiva se codifica mediante las feromonas acumuladas: las elecciones de una iteración alteran las probabilidades de las siguientes. En esta corrida A-B-Z dominó las elecciones; el resultado es empírico y no implica una garantía de selección en cada intento.

Código completo del punto 4

```python
"""Punto 4 - Modelo simplificado de ACO con tres rutas de A a Z."""
import random

RUTAS = {"A-B-Z": 6, "A-C-Z": 8, "A-D-Z": 10}
HORMIGAS = 10
ITERACIONES = 20
ALFA, BETA = 1, 2
RHO = 0.2
Q = 10
SEMILLA = 42


def probabilidades(feromonas):
    pesos = {r: (feromonas[r] ** ALFA) * ((1 / L) ** BETA) for r, L in RUTAS.items()}
    total = sum(pesos.values())
    return {r: w / total for r, w in pesos.items()}


def aco():
    random.seed(SEMILLA)
    tau = {r: 1.0 for r in RUTAS}
    frecuencias = {r: 0 for r in RUTAS}
    mejor_ruta, mejor_L = None, float("inf")
    nombres = list(RUTAS)
    for it in range(1, ITERACIONES + 1):
        p = probabilidades(tau)                                    # 1. probabilidades
        elecciones = random.choices(nombres, weights=[p[r] for r in nombres], k=HORMIGAS)
            # 2.
        for r in tau:                                              # 3. evaporacion
            tau[r] *= (1 - RHO)
        for r in elecciones:                                       # 4. deposito
            tau[r] += Q / RUTAS[r]
            frecuencias[r] += 1
            if RUTAS[r] < mejor_L:
                mejor_ruta, mejor_L = r, RUTAS[r]
        conteo = {r: elecciones.count(r) for r in nombres}
        print(f"It {it:2d}: p=" + ", ".join(f"{r}:{p[r]:.3f}" for r in nombres)
              + f" | elecciones={conteo} | tau=" + ", ".join(f"{r}:{tau[r]:.3f}" for r in
                  nombres))
    return frecuencias, tau, mejor_ruta, mejor_L

```

```python

if __name__ == "__main__":
    # Parte A
    d = {"B": 2, "C": 4, "D": 5}
    t = {"B": 1, "C": 2, "D": 1}
    w = {j: t[j] ** ALFA * (1 / d[j]) ** BETA for j in d}
    s = sum(w.values())
    for j in d:
        print(f"A->{j}: peso={w[j]:.4f}, p={w[j]/s:.4f}")
    nuevas = {j: (1 - RHO) * t[j] + (Q / 10 if j == "B" else 0) for j in d}
    print("Feromonas actualizadas:", nuevas, "\n")

    frec, tau, ruta, L = aco()
    total = sum(frec.values())
    print("\nFrecuencias de eleccion:")
    for r in RUTAS:
        print(f"  {r}: {frec[r]} ({frec[r]/total:.1%})")
    print("Feromonas finales:", {r: round(v, 4) for r, v in tau.items()})
    print(f"Mejor ruta observada: {ruta} (longitud {L})")
```

Listing 4: Implementación original del ACO simplificado

5. Punto 5. PSO: movimiento y memoria

5.1. A. Componente matemático y conceptual

Se minimiza $f(x) = x^2$ con una partícula en $x = 4$, velocidad $v = -1$, mejor personal $pbest = 2$, mejor global $gbest = 1$ y parámetros $w = 0{,}5$, $c_1 = c_2 = 1{,}5$, $r_1 = 0{,}4$, $r_2 = 0{,}6$.

5.1.1. A.1. Inercia, componente cognitivo y componente social

La actualización es $v' = wv + c_1 r_1(pbest - x) + c_2 r_2(gbest - x)$:

$$\underbrace{wv}_{\text{inercia}} = 0{,}5(-1) = \boxed{-0{,}5},$$

$$\underbrace{c_1 r_1(pbest - x)}_{\text{cognitivo}} = 1{,}5(0{,}4)(2 - 4) = \boxed{-1{,}2},$$

$$\underbrace{c_2 r_2(gbest - x)}_{\text{social}} = 1{,}5(0{,}6)(1 - 4) = \boxed{-2{,}7}.$$

Las tres contribuciones favorecen un desplazamiento hacia valores menores de x, y la de mayor magnitud es la social.

5.1.2. A.2. Nueva velocidad, posición y aptitud

$$v' = -0{,}5 - 1{,}2 - 2{,}7 = \boxed{-4{,}4}, \quad x' = 4 + (-4{,}4) = \boxed{-0{,}4}, \quad f(x') = (-0{,}4)^2 = \boxed{0{,}16}.$$

La nueva aptitud mejora la anterior, $f(4) = 16$, aunque el movimiento sobrepasa las posiciones de referencia *pbest* y *gbest*.

5.1.3. A.3. Actualización de *pbest* y *gbest*

Como $f(-0{,}4) = 0{,}16 < f(2) = 4$, corresponde actualizar $pbest' = -0{,}4$. Asimismo, $0{,}16 < f(1) = 1$, y suponiendo, como indica el enunciado, que las demás partículas no han mejorado, debe actualizarse $gbest' = -0{,}4$.

5.1.4. A.4. Mejor personal frente a mejor global

pbest es la mejor posición histórica de una partícula: registra su experiencia individual y determina el componente cognitivo. *gbest* es la mejor posición histórica conocida en el enjambre: representa la experiencia colectiva y determina el componente social. Cada partícula conserva su propio *pbest*, pero en la variante global de PSO comparte el mismo *gbest*.

5.1.5. A.5. Intercambio de información: PSO frente a ACO

En PSO, el intercambio ocurre a través de la posición global *gbest*, que influye directamente en las velocidades mediante el término social. En ACO, la comunicación es indirecta, por modificación del entorno: las hormigas depositan feromonas sobre las rutas y las siguientes utilizan esos rastros en sus probabilidades de elección. Ambos métodos aprovechan la experiencia del grupo, pero PSO representa el conocimiento compartido con una posición y ACO mediante intensidades de feromona distribuidas sobre alternativas.

5.2. B. Implementación de PSO en dos dimensiones

Se minimiza $f(x, y) = x^2 + y^2$ en $[-5, 5]^2$ mediante diez partículas inicializadas uniformemente con velocidad cero, $w = 0{,}5$, $c_1 = c_2 = 1{,}5$, treinta iteraciones y semilla 42. Los valores r_1, r_2 se regeneran independientemente por partícula, coordenada e iteración. Cada actualización de velocidad utiliza el mismo *gbest* disponible al comienzo de esa iteración; solo después de mover y evaluar el enjambre se actualizan memorias individuales y global. Se limitan las coordenadas fuera de $[-5, 5]$ y se anula exclusivamente la componente correspondiente de velocidad.

Tabla 7: Convergencia registrada del mejor valor global

Iteración	Mejor $f(x, y)$
0	7,760067
1	0,653273
5	0,067505
10	0,011169
15	0,00032619
20	$8{,}880881 \times 10^{-6}$
25	$5{,}964811 \times 10^{-7}$
30	$3{,}433969 \times 10^{-8}$

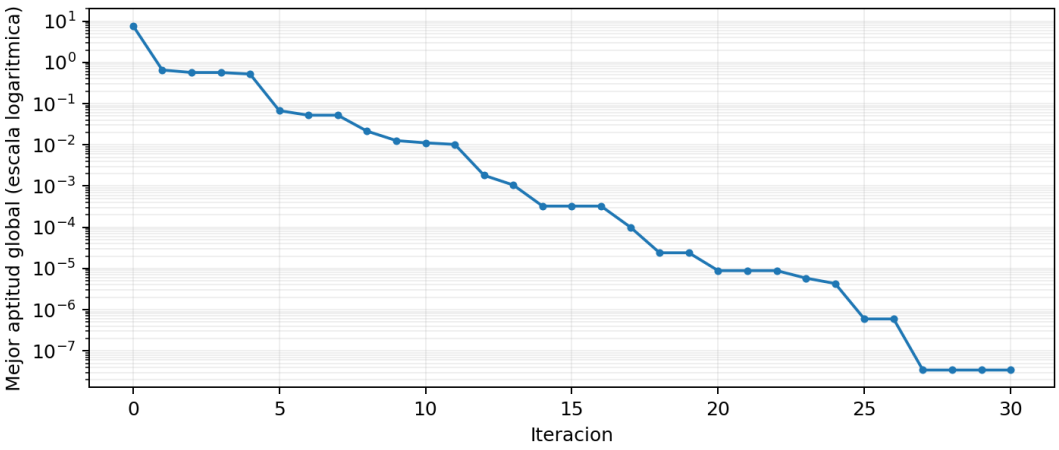


Figura 6: Evolución de la mejor aptitud global en escala logarítmica.

Al finalizar se obtuvo:

$$\boxed{gbest \approx (-0{,}000185,\ 0{,}000014)}, \qquad \boxed{f(gbest) \approx 3{,}433969 \times 10^{-8}}.$$

El mínimo teórico ocurre en $(0, 0)$ con valor 0, ya que $x^2 + y^2 \geq 0$. La ejecución se aproxima estrechamente a ese punto, pero no lo alcanza exactamente. Esto es compatible con movimientos continuos regidos por velocidades y factores aleatorios, un presupuesto finito de iteraciones y actualizaciones que no fuerzan a pasar por el origen. La mejor aptitud global acumulada no aumenta, porque *gbest* solo se sustituye al encontrar una mejora estricta.

Código completo del punto 5

```python
"""Punto 5 - PSO para minimizar f(x, y) = x^2 + y^2 en [-5, 5]^2."""
import numpy as np

N_PART = 10
DIM = 2
LIM_INF, LIM_SUP = -5.0, 5.0
W, C1, C2 = 0.5, 1.5, 1.5
ITERACIONES = 30
SEMILLA = 42


def f(pos):
    return np.sum(pos ** 2, axis=-1)


def pso():
    rng = np.random.default_rng(SEMILLA)
    x = rng.uniform(LIM_INF, LIM_SUP, size=(N_PART, DIM))
    v = np.zeros((N_PART, DIM))
    pbest = x.copy()
    pbest_val = f(x)
    g = np.argmin(pbest_val)
    gbest, gbest_val = pbest[g].copy(), pbest_val[g]
    historial = [gbest_val]
    print(f"Iter  0: gbest={gbest}, f={gbest_val:.6e}")

    for it in range(1, ITERACIONES + 1):
        r1 = rng.random((N_PART, DIM))   # nuevos por particula, dimension e iteracion
        r2 = rng.random((N_PART, DIM))
        # todas las particulas usan el mismo gbest de esta iteracion
        v = W * v + C1 * r1 * (pbest - x) + C2 * r2 * (gbest - x)
        x = x + v
        # ajuste al dominio: posicion al limite y velocidad de esa componente en cero
        fuera = (x < LIM_INF) | (x > LIM_SUP)
        x = np.clip(x, LIM_INF, LIM_SUP)
        v[fuera] = 0.0
        # evaluar y actualizar memorias
        val = f(x)
        mejora = val < pbest_val
        pbest[mejora] = x[mejora]
        pbest_val[mejora] = val[mejora]
        g = np.argmin(pbest_val)
        if pbest_val[g] < gbest_val:
            gbest, gbest_val = pbest[g].copy(), pbest_val[g]
        historial.append(gbest_val)
        print(f"Iter {it:2d}: gbest=({gbest[0]: .6f}, {gbest[1]: .6f}), f={gbest_val:.6e}")
    return gbest, gbest_val, historial


if __name__ == "__main__":
    # Parte A
    x, v, pb, gb = 4, -1, 2, 1
    inercia = 0.5 * v
    cognitivo = 1.5 * 0.4 * (pb - x)
    social = 1.5 * 0.6 * (gb - x)
    v_new = inercia + cognitivo + social
    x_new = x + v_new
    print(f"Inercia={inercia}, cognitivo={cognitivo:.2f}, social={social:.2f}")
```

```
    print(f"v_nuevo={v_new:.2f}, x_nuevo={x_new:.2f}, f={x_new**2:.4f}\n")

    gbest, val, hist = pso()
    print(f"\nMejor posicion final: ({gbest[0]:.6f}, {gbest[1]:.6f})")
    print(f"Valor de la funcion: {val:.6e}")
```

Listing 5: Implementación original de PSO bidimensional

Conclusiones generales

Los cinco métodos resuelven problemas de optimización mediante mecanismos diferentes. El algoritmo genético explora combinaciones de cromosomas con selección, cruce y mutación; ascenso de colinas aplica mejoras locales estrictas y depende del estado inicial; el temple simulado acepta probabilísticamente empeoramientos según su temperatura; ACO coordina elecciones mediante feromonas y evaporación; y PSO combina inercia, memoria individual y una referencia global. Las ejecuciones reproducidas verifican la implementación de los cinco apartados B. Las conclusiones experimentales corresponden a las semillas y parámetros exigidos y deben distinguirse de cualquier garantía matemática de convergencia.

Archivos y reproducibilidad

Este proyecto se compila mediante `main.tex` en Overleaf con `pdfLaTeX`. Contiene los cinco scripts originales en `codigo/`, las cinco trazas entregadas en `salidas/` y seis figuras en `figuras/`. Para ejecutar localmente:

```
python codigo/punto1_genetico.py
python codigo/punto2_ascenso_colinas.py
python codigo/punto3_temple_simulado.py
python codigo/punto4_aco.py
python codigo/punto5_pso.py
```

El último script requiere `numpy`. El informe hace explícita la diferencia entre los valores teóricos y las salidas producidas por una ejecución particular.